MODULE 6

NLP 與文字探勘：讓電腦讀懂文字

M6 — Natural Language Processing & Text Mining

護理與健康領域最有價值的資料，多半是文字：病歷、交班紀錄、
問卷開放題、衛教諮詢。這一週學怎麼把它們變成可以分析的東西。

資料科學　2027 Spring　·　第 11 週　·　Phase C　｜　謝楠楨

本講義大綱

01 為什麼要學這個 · 醫護領域大量資料是文字

02 NLP 的三個時代 · 規則式 → 統計 → 深度學習

03 文字探勘七步流程 · 跟 M1 資料工程的對照

04 前處理逐步實例 · 一句病歷的六個階段

05 ★ 中文斷詞 · 這一步做不好，後面全白做

06 詞袋與 DTM · 文字變成表格

07 TF-IDF · 為什麼「病人」這個詞沒有價值

08 N-gram 與否定偵測 · 把語序救回來

09 關鍵詞抽取 · 找出真正有訊息量的詞

10 文本分類 · Naive Bayes 手算實例

11 主題模型 · 從沒有標籤的文字找主題

12 情感分析 · 三種做法的實際差異

13 命名實體辨識 · 把敘述變成欄位

14 現代方法：零樣本與摘要 · 不用標註資料也能做

15 任務地圖 · 哪個任務該用哪一代方法

16 AI Studio 文字探勘實作 · 課堂範例

17 探索性文字分析 · 拿到資料的第一件事

18 文件相似度與標註品質 · 以及 Kappa 一致性

19 醫護應用案例 · 六個可執行的方向

20 醫療文本的特殊挑戰 · 否定、縮寫、去識別化

為什麼護理與健康科技的學生特別該學文字探勘

你手上最多的資料，其實是文字

- 護理紀錄、交班單、病程摘要、出院準備評估——全是文字
- 問卷的開放題、滿意度調查的意見欄、衛教諮詢的提問紀錄
- ★ 這些資料量大、價值高，但因為「不是數字」而長期被丟在一邊

文字資料的分析價值不輸結構化資料

- 護理紀錄裡的措辭變化，可能比生命徵象更早反映病情惡化
- 滿意度的星等只告訴你「幾分」，開放題才告訴你「為什麼」
- 跌倒與異常事件通報的敘述欄，藏著真正的根因

而且你有一個資工背景的人沒有的優勢

- ★ 你看得懂那些文字。你知道「主訴」「否認」「PRN」是什麼意思
- 主題模型跑出一組詞，資工的人不知道那代表什麼，你知道
- 領域知識在文字探勘裡的權重，比在數值分析裡更高

這一週的目標不是把你變成 NLP 工程師，而是讓你有能力判斷「這批文字能不能分析、該怎麼分析」。

實例：台灣病安通報——每季近 2 萬件，大多要靠人讀

數量：台灣病人安全通報系統（TPR，衛福部委託醫策會辦理）

- 2024 年第 2 季收案 18,152 件、第 3 季 19,500 件——每季近 2 萬件
- 前兩大類：藥物事件約 38%、跌倒事件約 23%（第 3 季分別 7,495 件、4,540 件）
- TPR 自己註明：自願匿名通報，數字只代表收案情況、不代表醫療現況——「資料怎麼來的」決定能下什麼結論（M9）

文字探勘能幫上什麼

- 文本分類：依事件經過的文字自動歸類，交叉檢查勾選的類別有沒有勾錯（PART 05）
- 主題模型：同樣是「跌倒」，再找出如廁、夜間下床、輔具等次主題，給改善方案用（PART 06）
- 否定偵測：「病人否認頭部撞擊」不能被算成頭部外傷（PART 04）

動手之前的現實

- 去識別化：通報內容含病人與人員資訊，分析前要先遮蔽（PART 09）
- 類別不平衡：嚴重傷害事件只佔少數，整體準確率會騙人（M2-2）
- 本講義的通報範例都是虛構；真實通報資料要經機構同意與倫理審查才能用

PART 01

NLP 的三個時代

70 Years of NLP

七十年來 NLP 換過三種完全不同的做法。
但新的沒有淘汰舊的——這一點對醫療場域特別重要。

NLP 的三個時代



★ 重點：新時代沒有淘汰舊時代

醫療場域今天仍大量使用規則式（如關鍵字警訊偵測）與 TF-IDF —— 因為它們可解釋、可稽核、不用算力。

同一個任務往往三代並用：規則式當安全網、傳統 ML 當主力、現代方法補委婉表達。

規則式（1950s-80s）

人工寫文法規則、關鍵字比對。
完全可解釋，但規則會爆炸。

統計與傳統 ML （1990s-2010s）

TF-IDF 、 Naive Bayes 、主題模型。
★ 這一週的主要內容。

深度學習（2013- 今）

Word2Vec → BERT → GPT 。懂上下文，但不易解釋。
M7 會深入。

★ 新的沒有淘汰舊的

醫療今天仍大量使用規則式（關鍵字警訊偵測）
與 TF-IDF—— 因為可解釋、可稽核、不用算力。

原講義： GIA 2. NLP - Natural Language Processing.pptx pp.9–13 ★原生圖形，可直接在 PowerPoint 編輯

同一個任務「找出有自殺意念的諮詢紀錄」，三代做法對照

做法	怎麼做	優點	問題
規則式	建關鍵字表：「不想活」「結束一切」「沒有意義」等，命中就標記	★ 完全可稽核；漏掉哪個詞看得出來；零成本	換句話說就抓不到；「我不想活在這種痛裡」與 「我不想活了」無法區分
傳統 ML	把標註過的紀錄做成 TF-IDF，訓練分類器	能學到關鍵字表沒想到的詞；可算出各詞的權重	需要幾百筆標註資料；仍看不懂語序與否定
現代方法	用預訓練模型做零樣本或少樣本分類	不需要（或只需要極少）標註；理解上下文與委婉表達	★ 說不清楚為什麼判成這一類——在高風險場景是致命問題

★ 實務常見做法：三者並用。規則式當第一層安全網（保證不漏關鍵詞），模型當第二層（抓委婉表達），人工覆核當第三層。

實例：規則式的極限——用關鍵字找「跌倒相關」通報

虛構通報內容	真的是跌倒?	規則 1：「跌倒」「滑倒」	規則 2：再加「摔」「倒臥」「坐在地上」	規則 3：再排除否定／衛教語境
病人下床如廁時滑倒，右膝擦傷	是	✓ 抓到	✓ 抓到	✓ 抓到
夜班巡房發現病人倒臥浴室地板	是	✗ 漏掉	✓ 抓到	✓ 抓到
病人自述腳軟，坐在地上等待協助	是	✗ 漏掉	✓ 抓到	✓ 抓到
地板濕滑已立警示牌，未有人跌倒	否	✗ 誤報	✗ 誤報	✓ 排除
衛教家屬預防跌倒注意事項	否	✗ 誤報	✗ 誤報	✓ 排除
病人從輪椅摔下，頭部無外傷	是	✗ 漏掉	✓ 抓到	✓ 抓到
病人自述差點跌倒，扶住床欄未倒地	否	✗ 誤報	✗ 誤報	✓ 排除
病人主訴頭暈，協助臥床休息	否	✓ 排除	✓ 排除	✓ 排除
合計（4 件真的是跌倒）		抓到 1　漏 3　誤報 3	抓到 4　漏 0　誤報 3	抓到 4　漏 0　誤報 0

換個說法就抓不到；加了關鍵字又把「未有人跌倒」「預防跌倒」「差點跌倒」當成跌倒；再加否定規則才全對——但這只是 8 筆。真實資料會一直出現新說法，規則越寫越多（規則爆炸）。實務上規則當安全網、模型抓委婉表達、人工覆核當最後一層。

PART 02

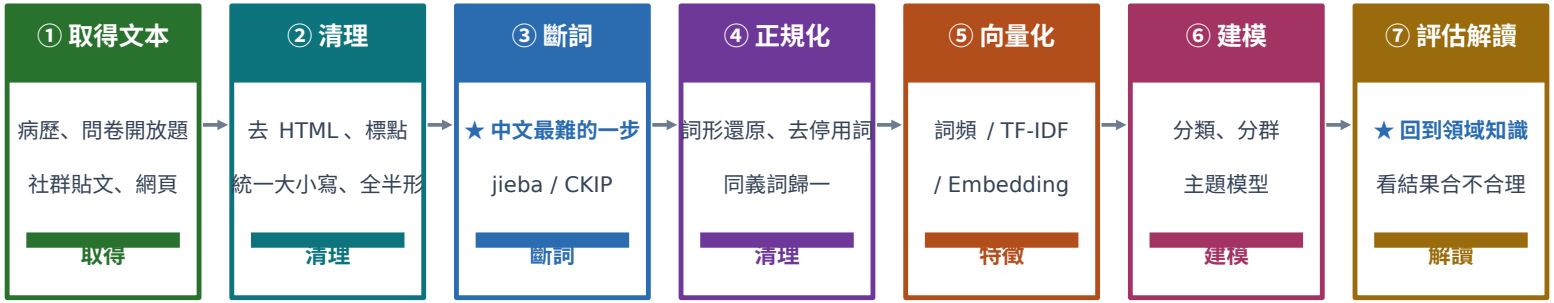
文字探勘的完整流程

The Text Mining Pipeline

七個步驟。其中大部分時間會花在前幾步。

這不是誇飾——中文尤其如此。

七步流程，以及跟 M1 的對照



跟 M1 的資料工程流程對照著看

- ②③④ 就是 M1 的「資料清理」——只是對象從數字欄位換成文字
- ⑤ 向量化 ＝ M1 的「特徵工程」——把非結構化文字變成模型吃得下的矩陣
- ⑥⑦ 之後接的就是 M2 ／ M3 學過的分類器與分群法，完全沿用

★ 大部分時間花在 ②③④——中文文本尤其如此。

很多人以為文字探勘的難點在演算法，其實難點在「把字變乾淨」。演算法反而是現成的。

這也是為什麼 M1 的資料工程觀念在這一週會一直出現。

②③④ ＝ M1 的資料清理

只是對象從數字欄位換成文字。

⑤ 向量化 ＝ M1 的特徵工程

把非結構化文字變成模型吃得下的矩陣。
★ 這是整個流程的核心轉換。

⑥⑦ 完全沿用 M2 ／ M3

向量化之後接的就是你已經學過的分類器與分群法。
文字探勘沒有新的演算法。

★ 難點不在演算法

難點在「把字變乾淨」。演算法反而是現成的。

原講義： GIA 2. NLP - Natural Language Processing.pptx　pp.19–21　★ 原生圖形，可直接在 PowerPoint 編輯

前處理逐步實例：一句病歷的六個階段

原始文本	病人 78 歲，主訴「胸痛」三天，於 2027/3/5 就診。 否認發燒。 從系統匯出的原始字串，含 HTML 標籤與全形符號
① 清理	病人 78 歲 主訴 胸痛 三天 於 2027/3/5 就診 否認發燒 去 HTML 、去標點、全形轉半形
② 斷詞	病人｜ 78 ｜歲｜主訴｜胸痛｜三天｜於｜ 2027/3/5 ｜就診｜否認｜發燒 **★ 中文沒有空格，這一步決定後面所有結果**
③ 去停用詞	病人｜ 78 ｜歲｜主訴｜胸痛｜三天｜就診｜否認｜發燒 移除「於」等無實質意義的詞
④ 正規化	病人｜歲｜主訴｜胸痛｜天｜就診｜否認｜發燒 ＋ [年齡 :78] [日期 :2027-03-05] 數字與日期抽成結構化欄位，不留在詞袋裡
⑤ 向量化	{ 病人 :1, 主訴 :1, 胸痛 :1, 就診 :1, 否認 :1, 發燒 :1, ...} **★ 注意：「否認發燒」被拆成兩個詞，語意已經丟失**

★ 最後一列是醫療 NLP 的核心風險

詞袋模型丟掉語序之後，「否認發燒」與「發燒」在向量上幾乎沒有差別。

解法在 PART 4 （N-gram）與否定範圍標記——這是醫療文本最該補的一課。

從系統匯出的原始字串

含 HTML 標籤、全形符號、日期、數字——都要處理。

★ 斷詞是分水嶺

中文沒有空格。這一步決定後面所有結果。

數字與日期要抽出來

年齡、日期、檢驗值應該變成結構化欄位，不要留在詞袋裡當成一般的詞。

★ 最後一列是核心風險

詞袋丟掉語序後，「否認發燒」與「發燒」在向量上幾乎沒有差別。

原講義： GIA 2. NLP - Natural Language Processing.pptx pp.23–30 ★ 原生圖形，可直接在 PowerPoint 編輯

前處理的六個技巧：做什麼、什麼時候不要做

技巧	做什麼	什麼時候不要做
轉小寫 **Lowercasing**	統一大小寫，避免 Fever 與 fever 被當成兩個詞	★ 需要辨識縮寫時（US 美國 vs us 我們）；藥名大小寫有意義時
去標點 **Punctuation Removal**	移除標點符號	做情感分析時（「!!!」有意義）； 要保留句界時
斷詞 **Tokenization**	把文字切成詞	★ 從不跳過。但中文要慎選工具與詞典
去停用詞 **Stop Words**	移除「的」「了」「是」等無意義的詞	★ 做否定偵測時（「不」「無」「否認」常在停用詞表裡！）
詞形還原 **Lemmatization**	running → run，把詞還原成原形	中文沒有詞形變化，這一步主要用於英文
詞性標註 **POS Tagging**	標出名詞、動詞等，可用來篩選	多數任務不需要；想只看名詞做主題分析時才用

★ 第四列是醫療文本最容易踩的坑：預設的中文停用詞表通常包含否定詞。照抄別人的停用詞表，你的模型會分不出「有」跟「沒有」。

PART 03

中文斷詞

Chinese Word Segmentation

英文天生有空格，中文沒有。

這一個差別，讓中文 NLP 的第一步就比英文難上一個量級。

中文斷詞的難題與兩個主要工具

英文天生有空格；中文沒有。同一串字可以有多種切法，而切法不同結果就完全不同。

「乾女兒」	乾｜女兒　vs　乾女兒	一般詞彙的歧義
「研究生命的起源」	研究生｜命的起源　vs　研究｜生命｜的｜起源	★ 經典例子，切錯語意全變
「病人服藥後不適」	病人｜服藥｜後｜不適　vs　病｜人服｜藥後｜不適	醫療文本的斷詞風險
「二尖瓣脫垂」	二｜尖｜瓣｜脫垂　vs　二尖瓣｜脫垂	★ 專有名詞未收錄就會被切碎

jieba （結巴）

- 簡體語料為主（可換繁體詞典），台灣用語較弱
- 輕量、快、易加自訂詞典
- 三種模式：精確／全模式／搜尋引擎
- 適合：社群貼文、非正式文本、需頻繁加詞

CKIP Transformers （中研院）

- **★ 以臺灣繁體語料訓練，本地用語準確**
- 同時提供詞性標註與命名實體辨識
- 較重、需下載模型，速度較慢
- **適合：★ 醫療、公文、學術等嚴謹文本**

★ 醫療文本的關鍵動作：把專科術語做成自訂詞典餵給斷詞器

不做這件事，「二尖瓣脫垂」會被切成四個沒意義的字，後面所有分析都建立在錯的詞上。

歧義切分

「研究生命的起源」可以切成「研究生｜命的起源」，
也可以切成「研究｜生命｜的｜起源」。

jieba （結巴）

以簡體語料為主（可換繁體詞典），台灣用語較弱。輕量、快、易加詞典。
適合社群貼文等非正式文本。

★ CKIP Transformers （中研院）

以臺灣繁體語料訓練，本地用語準確，
同時提供詞性與命名實體。適合醫療、公文等嚴謹文本。

★ 最關鍵的動作

把專科術語做成自訂詞典。不做這件事，
「二尖瓣脫垂」會被切成四個沒意義的字。

★ 原生圖形，可直接在 PowerPoint 編輯

實作：中文斷詞與自訂詞典

重點在最後那段——自訂詞典帶來的差異

Python （課後自學；課堂以 AI Studio 為主）

```
# ── 方法一：jieba（輕量、快）──────────────────────────────────
import jieba

text = '病人主訴二尖瓣脫垂合併心房顫動，否認胸痛'
print('/'.join(jieba.cut(text)))
#  病人 / 主訴 / 二 / 尖 / 瓣 / 脫 / 垂 / 合併 / 心房 / 顫動 / ， / 否認 / 胸痛  （示意，依版本而異）
#          ↑ 專有名詞被切碎了

# ★ 加入自訂詞典後
jieba.load_userdict('medical_terms.txt')   #  一行一個詞
print('/'.join(jieba.cut(text)))
#  病人 / 主訴 / 二尖瓣脫垂 / 合併 / 心房顫動 / ， / 否認 / 胸痛
#          ↑ 正確了

# ── 方法二：CKIP Transformers（繁中準確度較佳）──────────────────
from ckip_transformers.nlp import CkipWordSegmenter, CkipPosTagger

ws  = CkipWordSegmenter(model='bert-base')
pos = CkipPosTagger(model='bert-base')
words = ws([text]); tags = pos(words)
#  同時得到斷詞與詞性，且對臺灣用語與繁體較準

# medical_terms.txt  格式：一行一個詞（可加詞頻與詞性）
#    二尖瓣脫垂 100 n /  心房顫動 100 n /  慢性阻塞性肺病 100 n
```

實例：同一句病歷，有沒有自訂詞典差很多

	只用基本詞典	加入 7 個醫療術語的自訂詞典
病人主訴二尖瓣脫垂合併心房顫動，否認胸痛	病人 | 主訴 | 二 | 尖 | 瓣 | 脫垂 | 合併 | 心房 | 顫動 | ， | 否認 | 胸痛	病人 | 主訴 | **二尖瓣脫垂** | 合併 | **心房顫動** | ， | 否認 | 胸痛
慢性阻塞性肺病病人需要鼻胃管灌食	慢性 | 阻塞 | 性 | 肺病 | 病人 | 需要 | 鼻 | 胃管 | 灌食	**慢性阻塞性肺病** | 病人 | 需要 | **鼻胃管** | 灌食
病人有壓力性損傷，傷口滲液增加	病人 | 有 | 壓力 | 性 | 損傷 | ， | 傷口 | 滲液 | 增加	病人 | 有 | **壓力性損傷** | ， | 傷口 | 滲液 | 增加
糖化血色素偏高，注意低血糖	糖化 | 血色素 | 偏高 | ， | 注意 | 低 | 血糖	**糖化血色素** | 偏高 | ， | 注意 | **低血糖**
經典歧義：研究生命的起源	正向最大匹配：研究生 | 命 | 的 | 起源	逆向最大匹配：研究 | 生命 | 的 | 起源

紅底＝被切成單字的可疑片段（「二尖瓣脫垂」→ 二｜尖｜瓣｜脫垂）；綠底＝自訂詞典命中。後面的 TF-IDF 、主題模型都建立在這些詞上——斷詞錯了，後面全白做。（簡化的最大匹配法示範； jieba 、 CKIP 會用統計或模型選切法）

建立醫療自訂詞典的四個來源（上）

① 現成的標準詞表

- ICD-10 診斷碼的中文名稱、健保藥品品項檔、 LOINC 檢驗項目
- 衛福部與各學會公布的名詞對照表
- ★ 這些多半可公開下載，是最省力的起點

② 從你自己的資料中挖

- 先用預設詞典跑一次，把「被切成單字的連續序列」找出來人工檢視
- 常出現的字串組合往往就是漏掉的專有名詞
- 統計方法：互資訊（ PMI ）與左右鄰字熵可自動偵測新詞

這兩個來源可以讓你在半天內建出一份堪用的詞典。

建立醫療自訂詞典的四個來源（下）

③ 科內的慣用語與縮寫

- 每個單位都有自己的簡寫： BP 、 I/O 、 NPO 、 on Foley
- ★ 這些外面的詞典絕對不會有，只能你自己整理
- 順帶把同義詞歸一：「高血壓／ HTN ／ hypertension 」對到同一個詞

④ 迭代維護

- 詞典不是做一次就好。新藥、新術式、新縮寫會一直出現
- 把它當成要持續維護的資產，而不是一次性的前處理
- 建議：每次分析發現被切碎的詞就補進去，累積成科內資產

★ 一份好的自訂詞典，對結果的提升通常大於換一個更厲害的模型。這是文字探勘最划算的投資。

PART 04

向量化：文字變成數字

Vectorization

整個文字探勘流程的核心轉換就在這裡。

轉完之後，你就回到 M2 ／ M3 熟悉的世界了。

Document-Term Matrix：文字變成表格

語料：四篇病歷（已斷詞）

D1　病人 主訴 胸痛 三 天

D2　病人 主訴 頭痛 兩 天

D3　病人 否認 胸痛 但 有 頭痛

D4　護理 指導 衛教 出院 用藥

詞 →	病人	主訴	胸痛	頭痛	天	否認	護理	衛教
D1	1	1	1	0	1	0	0	0
D2	1	1	0	1	1	0	0	0
D3	1	0	1	1	0	1	0	0
D4	0	0	0	0	0	0	1	1

稀疏 Sparsity

這個 4 × 8 的小矩陣，32 格中就有 18 格是 0（56%）

真實資料是幾千篇 × 幾萬個詞 → 99% 以上是 0

順序沒了 Order Lost

「病人主訴胸痛」與「胸痛主訴病人」→ 同一個向量

「否認胸痛」的「否認」也只是袋子裡的一顆球

★ 這叫「詞袋」（Bag of Words）——因為它真的只是一個袋子，裡面的詞沒有順序

格子裡的數字是「這個詞在這篇文件出現幾次」；空白（0）代表沒出現。整個文字探勘的核心轉換就是這一步。

每一列是一份文件

每一欄是一個詞，格子裡是次數。

這叫「詞袋」 Bag of Words

因為它真的只是一個袋子——裡面的詞沒有順序。

★ 詞袋最大的限制

「病人主訴胸痛」與「胸痛主訴病人」在矩陣裡完全相同。
語序資訊全部丟失。

矩陣會很稀疏

真實資料通常是幾千篇 × 幾萬個詞，
而每篇只用到其中幾十個 → 99% 以上是 0 。

原講義： GIA 2. NLP - Natural Language Processing.pptx　pp.32　★ 原生圖形，可直接在 PowerPoint 編輯

TF-IDF：為什麼「病人」這個詞沒有價值

同一份文件 D1 「病人 主訴 胸痛 三 天」：兩種權重的差別（語料共四篇）

詞頻（Count）

每個詞都是 1 —— 看不出誰重要

		出現篇數 df
三	1	1 / 4 篇
主訴	1	2 / 4 篇
胸痛	1	2 / 4 篇
天	1	2 / 4 篇
病人	1	3 / 4 篇

結論：五個詞一樣重要（顯然不對）

TF-IDF

稀有詞被加權、常見詞被壓低

		出現篇數 df
三	0.553	1 / 4 篇
主訴	0.436	2 / 4 篇
胸痛	0.436	2 / 4 篇
天	0.436	2 / 4 篇
病人	0.353	3 / 4 篇

結論：「三」最能代表 D1 、「病人」貢獻最小

★ IDF 在做的事：出現在越少篇文件裡的詞，越能代表這篇文件

「病人」出現在 4 篇中的 3 篇 → 沒有鑑別力 → 權重最低（0.353）。

「三」只出現在 1 篇 → 最能代表這份文件 → 權重最高（0.553）。

★ 但「三」是數字，不該當關鍵詞 —— 這正是前處理第 ④ 步要先把數字抽成欄位的理由。

問題

詞頻把每個詞當成一樣重要。但「病人」出現在每一篇病歷裡，
它完全沒有鑑別力。

TF　詞頻

這個詞在這篇文件出現幾次（除以文件長度做正規化）。

IDF　逆文件頻率

出現在越少篇文件中的詞，權重越高。
公式（scikit-learn 版）： ln((1+N)/(1+df)) + 1 。

★ 結果

「病人」 4 篇中出現 3 篇 → 權重 0.353（最低）；
「三」只出現 1 篇 → 權重 0.553 （最高）。

原講義： GIA 2. NLP - Natural Language Processing.pptx　pp.36–39　★ 原生圖形，可直接在 PowerPoint 編輯

TF-IDF 完整手算：四篇病歷

所有數值皆以 scikit-learn 的公式實際計算並驗證

手算範例（已用 numpy 驗算）

```
【語料】四篇（已斷詞）
 D1   病人 主訴 胸痛 三 天
 D2   病人 主訴 頭痛 兩 天
 D3   病人 否認 胸痛 但 有 頭痛
 D4   護理 指導 衛教 出院 用藥

【IDF 計算】 idf(t) = ln( (1+N) / (1+df) ) + 1     N = 4

詞   df   idf
病人  3   ln(5/4)+1 = 1.223      ← 三篇都有，權重最低
主訴  2   ln(5/3)+1 = 1.511
胸痛  2   ln(5/3)+1 = 1.511
頭痛  2   ln(5/3)+1 = 1.511
天   2   ln(5/3)+1 = 1.511
三   1   ln(5/2)+1 = 1.916      ← 只有一篇有，權重最高

【D1 的向量】 tf × idf，再做 L2 正規化

三   1 × 1.916 = 1.916  →  0.553   ← 最能代表 D1 的詞
主訴 1 × 1.511 = 1.511  →  0.436
胸痛 1 × 1.511 = 1.511  →  0.436
天   1 × 1.511 = 1.511  →  0.436
病人 1 × 1.223 = 1.223  →  0.353   ← 貢獻最小
```

★ 注意「三」變成最重要的詞——這其實是個問題（數字不該當關鍵詞）。
所以前處理時要先把數字抽成結構化欄位。這就是前面第 ④ 步的用意。

三種向量化方式的比較

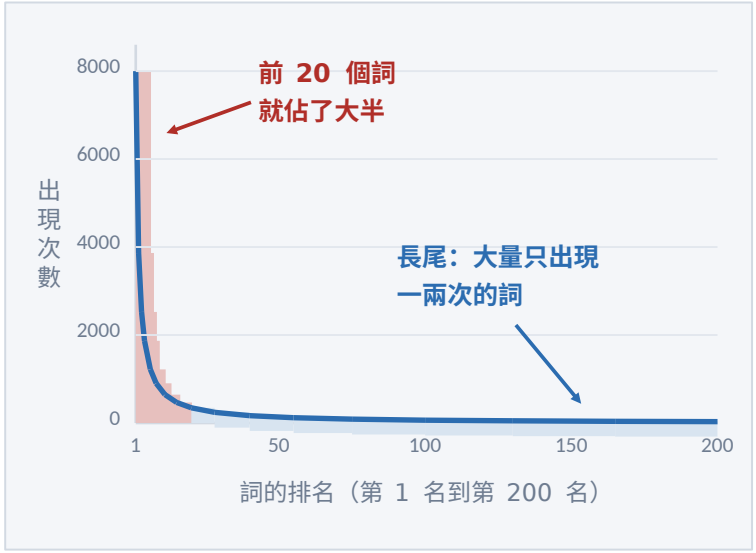
方式	怎麼算	保留什麼	丟掉什麼	什麼時候用
詞頻 Count	數次數	詞的出現與次數	語序、語意、詞的重要性差異	文件很短、詞彙很受控時
TF-IDF	詞頻 × 逆文件頻率	詞的相對重要性	語序、語意	★ 傳統文字探勘的預設選擇
嵌入 Embedding	用預訓練模型轉成稠密向量	★ 語意（同義詞會靠近）＋部分上下文	可解釋性	需要語意相似、或詞彙變化大時（M7 深入）

★ 一個實用建議：先用 TF-IDF 跑一次當基準線。如果 TF-IDF ＋ 邏輯斯迴歸就夠好，就不需要動用嵌入與深度模型。

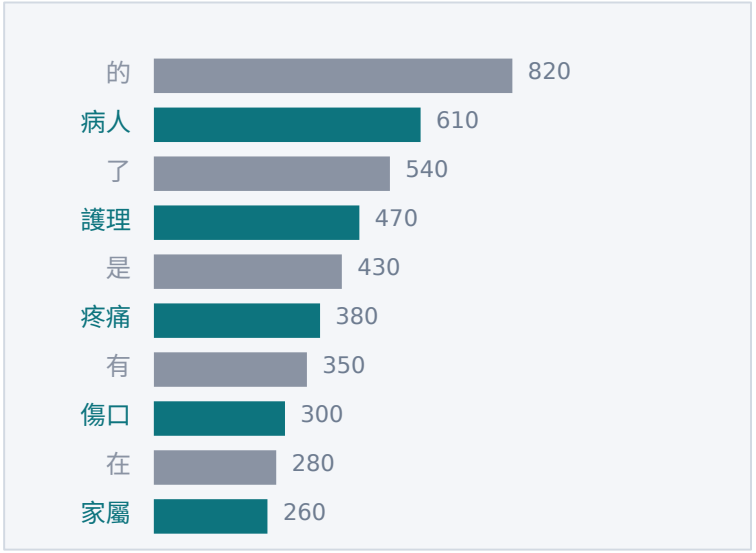
探索性文字分析：先看詞頻長什麼樣

這是拿到一批文字之後該做的第一件事

① 詞頻分布：極度不平均（Zipf 定律）



② 去停用詞前：前十名有一半是廢話



灰色＝停用詞（沒有訊息量）　青色＝有意義的詞　★ 前十名有一半是停用詞

★ 兩個實務啟示

① 不去停用詞，你的分析結果會被「的、了、是」淹沒 —— 每個主題長得一模一樣。

② 長尾的詞出現太少，統計上不可靠：實務上會設「最少出現 3 次」的門檻把它們濾掉。

Zipf 定律

詞頻極度不平均：少數幾個詞佔掉大半，剩下是一條長尾。所有自然語言都是這樣。

★ 啟示一

不去停用詞，你的分析結果會被「的、了、是」淹沒。

★ 啟示二

長尾的詞出現太少，統計上不可靠。
實務上會設「最少出現 3 次」的門檻把它們濾掉。

動手第一步

任何文字探勘專案，先畫這兩張圖。
看不出問題就先別急著建模。

★ 原生圖形，可直接在 PowerPoint 編輯

探索階段該看的六個指標

指標	怎麼看	看出什麼
文件數與長度分布	畫長度直方圖	有沒有極短（空白）或極長（複製貼上）的異常紀錄
詞彙量（vocabulary size）	去重後的詞數	太小 → 用語高度制式；太大 → 可能斷詞有問題
前 50 高頻詞	長條圖	★ 停用詞夠不夠乾淨；有沒有該加進詞典的術語
只出現 1-2 次的詞佔比	計數	通常佔 50% 以上。這些多半是雜訊或錯字
共現詞對（bigram）	統計相鄰詞對	★ 找出被切碎的專有名詞的最好方法
各類別的詞頻差異	分組比較	哪些詞真的能區分類別——建模前的直覺

★ 第五列特別實用：如果「二尖瓣」與「脫垂」總是相鄰出現，那它們八成該合成一個詞。

N-gram： 把語序部分救回來

原句：「病人 否認 胸痛」（已斷詞，三個詞）

Unigram （1-gram）

病人 ／ 否認 ／ 胸痛

3 個特徵。「否認」與「胸痛」各自獨立，看不出「否認」是在修飾「胸痛」。

Bigram （2-gram）

病人否認 ／ 否認胸痛

★「否認胸痛」變成一個特徵了 —— 這正是我們要的。

Trigram （3-gram）

病人否認胸痛

整句變成一個特徵。太過細碎 —— 換個主詞就對不上，容易過擬合。

好處

- 部分挽回語序資訊
- **★ 否定詞、程度詞的修飾關係救回來了**
- 常見搭配詞（collocation）被保留
- 對情感分析與否定偵測提升明顯

代價

- 特徵數暴增： 1 萬詞 → 1 億種 bigram
- 矩陣更稀疏，多數 bigram 只出現一次
- 訓練變慢、記憶體吃緊
- **★ 實務：只保留出現 ≥ 5 次的 bigram**

★ 實務建議： unigram + bigram （AI Studio： Generate n-Grams），再設最低出現次數門檻

這是投報率最高的設定：幾乎不用改程式，就把否定與常見搭配救回來一大半。

問題回顧

詞袋丟掉語序，「否認胸痛」被拆成兩個無關的詞。

★ Bigram 的價值

「否認胸痛」變成一個特徵——這正是醫療文本需要的。

代價

特徵數暴增、矩陣更稀疏、訓練變慢。

★ 實務設定

unigram + bigram （AI Studio： Generate n-Grams），
再設「最低出現 5 次」的門檻。投報率最高的組合。

★ 原生圖形，可直接在 PowerPoint 編輯

三種挽救語序的方法，由輕到重

方法	做什麼	成本	效果
N-gram	把相鄰的 2–3 個詞合成一個特徵	低（改一個參數）	★ 能救回否定與常見搭配，多數場景夠用
否定範圍標記	偵測否定詞，把其後 N 個詞加上前綴（胸痛 → NEG_ 胸痛）	中（要寫規則）	★ 醫療文本效果顯著； NegEx 是經典做法
語法依存分析	解析句子結構，找出修飾關係	高（要模型與算力）	最精確，但中文的依存分析仍不夠穩定
改用現代模型	Transformer 天生理解語序	高（要 GPU 或 API）	最好，但失去可解釋性（M7 深入）

★ 順序建議：先加 bigram （免費）→ 不夠再加否定標記（便宜）→ 還不夠才考慮換模型。

否定偵測：醫療文本最該做的一件事

十行規則，換來分類品質的明顯提升

Python （原理示範；實務可用 negspacy 等套件）

```
# ── 概念: NegEx 風格的否定範圍標記 ──────────────────────────────
NEG_CUES = ['否認', '無', '未', '沒有', '不曾', '排除', '非']
STOP_MARK = [',', ',', '。', '但', '然而', '不過']   # 否定範圍的邊界
def mark_negation(tokens, window=5):
    out, neg_left = [], 0
    for t in tokens:
        if t in NEG_CUES:
            neg_left = window            # 啟動否定範圍
            out.append(t); continue
        if t in STOP_MARK:
            neg_left = 0                 # 遇到邊界就關閉
            out.append(t); continue
        if neg_left > 0:
            out.append('NEG_' + t)       # ★ 加前綴，變成不同的特徵
            neg_left -= 1
        else:
            out.append(t)
    return out
# ── 效果 ──────────────────────────────────────────────────────
mark_negation(['病人','否認','胸痛',',', ',','主訴','頭痛'])
# → ['病人', '否認', 'NEG_胸痛', ',', ',', '主訴', '頭痛']
#                  ↑ 現在「NEG_胸痛」與「胸痛」是兩個不同的特徵了

# ★ STOP_MARK 很重要: 沒有邊界，「否認胸痛，主訴頭痛」會把
#    「頭痛」也標成否定 —— 那就完全錯了。
```

實例：否定範圍標記——標到哪裡為止？

✓ 遇到標點或轉折詞就關閉否定範圍

病人 否認 NEG_胸痛 , 主訴 頭痛

無 NEG_發燒 , 但 咳嗽 喘

未 NEG_見 NEG_傷口 NEG_紅腫 NEG_滲液 , 疼痛 加劇

排除 NEG_肺炎 NEG_可能 , 持續 觀察 血氧

病人 表示 頭暈 緩解

✗ 沒有邊界（視窗 5 個詞）

病人 否認 NEG_胸痛 , NEG_主訴 NEG_頭痛

無 NEG_發燒 , 但 NEG_有 NEG_咳嗽 NEG_及 NEG_喘

未 NEG_見 NEG_傷口 NEG_紅腫 NEG_滲液 , NEG_疼痛 加劇

排除 NEG_肺炎 NEG_可能 , NEG_持續 NEG_觀察 NEG_血氧

病人 表示 頭暈 緩解

✗ 順序反了：
先去停用詞再標否定

發燒 , 但 咳嗽 喘

否定詞「無」被當成停用詞刪掉，「發燒」就變成有發燒。要先做否定標記，或把否定詞從停用詞表移除。

橘＝否定詞、紅＝被標成否定（NEG_）、灰＝邊界。沒有邊界時，「否認胸痛，主訴頭痛」的頭痛也被標成沒有。「已緩解」這類隱含否定規則抓不到——建議從自己的語料統計否定說法，不要只抄現成清單。

否定偵測的三個容易出錯的地方

① 否定詞本身常被停用詞表移除

- 中文停用詞表幾乎都包含「不」「無」「沒」
- ★ 先做否定標記，再去停用詞——順序反了就前功盡棄
- 或者：從停用詞表中把否定詞刪掉

② 否定範圍要設邊界

- 「否認胸痛，主訴頭痛」——「頭痛」不該被標成否定
- 標點、轉折詞（但、然而）都是天然的邊界
- 視窗大小（window）建議 3–5 個詞，太長容易誤標

③ 中文的否定表達比想像中多樣

- 「否認」「無」「未見」「排除」「不曾」「並無」「查無」
- ★ 還有隱含否定：「已改善」「已緩解」「恢復正常」
- 建議：從你自己的語料中統計，不要只抄現成清單

關鍵詞抽取：找出真正有訊息量的詞

同一批病人滿意度留言，兩種排序的天壤之別（各取前 10 名）

① 只看詞頻：全是「本來就會出現」的詞



② TF-IDF 加權：真正有訊息量的詞浮出來



★ 左邊那些詞每一則留言都有，看了等於沒看；右邊才是可以拿去開會討論的東西

做法：對每個類別（或每個時間區間）各算一次 TF-IDF，取前 10 名。

延伸：比較「本月」與「上月」的關鍵詞差異，就看得出問題的變化趨勢。

只看詞頻的問題

跑出來的全是「住院、醫師、護理師」——每一則留言都有，看了等於沒看。

★ TF-IDF 加權之後

「動線混亂、領藥、停車、候診椅、叫號」浮出來了。
這才是可以拿去開會討論的東西。

做法

對每個類別（或每個時間區間）各算一次 TF-IDF，取前 10 名。

延伸

比較「本月」與「上月」的關鍵詞差異，可以看出問題的變化趨勢。

★ 原生圖形，可直接在 PowerPoint 編輯

四種關鍵詞抽取方法

方法	原理	優點	限制
詞頻	數次數	最簡單	★ 幾乎沒用，全是常見詞
TF-IDF	相對於整體語料的稀有度	★ 簡單有效，本課程主力	看不出詞與詞的關係
TextRank	把詞當節點、共現當邊，跑 PageRank	考慮詞之間的關聯，單一文件也能用	計算較慢
KeyBERT ／嵌入式	用語意向量找出最能代表全文的詞	理解語意，能抓到換句話說的表達	需要模型與算力（ M7 ）

★ 一個實用組合： TF-IDF 抓出候選詞 → 人工快速篩一遍 → 得到的清單可以直接當自訂詞典的補充來源。

PART 05

文本分類

Text Classification

向量化之後， M2-2 學過的所有分類器都能直接用。
但有一個演算法特別適合文字——而且它「天真」得可愛。

Naive Bayes 手算實例

訓練資料（10 封信）

垃圾信 spam： 4 封 ｜ 正常信 ham： 6 封

「免費」出現在： spam 3 封、 ham 1 封

「會議」出現在： spam 0 封、 ham 4 封

待判斷的新信件

「限時免費領取，點擊連結」

含「免費」○　　不含「會議」 ×

計算（含 Laplace 平滑，避免機率為 0）

先驗機率	P(spam) = 4/10 = 0.4　　P(ham) = 6/10 = 0.6
條件機率	P(免費 |spam) = (3+1)/(4+2) = 0.667　　P(免費 |ham) = (1+1)/(6+2) = 0.250
	P(會議 |spam) = (0+1)/(4+2) = 0.167　　P(會議 |ham) = (4+1)/(6+2) = 0.625
相乘	spam 分數 = 0.4 × 0.667 × (1−0.167) = 0.2222
	ham 分數 = 0.6 × 0.250 × (1−0.625) = 0.0563
正規化	P(spam| 這封信) = 0.2222 / (0.2222+0.0563) = 0.798

判定： 79.8% 是垃圾信 → 分類為 spam

★「Naive （天真）」指的是：它假設「免費」與「會議」互相獨立——這在語言上明顯不成立，但它照樣好用。

原理

貝氏定理：看到這些詞，這封信是垃圾信的機率有多高。

Laplace 平滑

「會議」在 spam 中出現 0 次 → 機率為 0 → 整個乘積歸零。
分子加 1 、分母加 2 就避開了這個問題。

結果

P(spam | 含「免費」不含「會議」) = 0.798 → 判為垃圾信。

★ 為什麼叫「天真」

它假設「免費」與「會議」互相獨立。
語言上這明顯不成立——但它照樣好用。

原講義： GIA 2. NLP - Natural Language Processing.pptx　pp.51–56　★ 原生圖形，可直接在 PowerPoint 編輯

該選哪個分類器？依資料量決定

資料量	首選	備選	為什麼
小（< 1 萬筆）	★ Naive Bayes	邏輯斯迴歸、 kNN	參數少、不易過擬合、訓練極快；文字資料上表現意外地好
中（1 萬– 100 萬筆）	邏輯斯迴歸	決策樹、隨機森林、梯度提升樹	★ 邏輯斯迴歸可看係數 → 知道是哪些詞造成判斷，醫療場域重要
大（> 100 萬筆）	線性 SVM ／邏輯斯迴歸	梯度提升樹	線性模型最擅長大量稀疏特徵，訓練快又穩
沒有標註資料	零樣本分類（ M7 ）	主題模型 ＋ 人工命名	★ 這是最常見的真實情況：資料很多，但沒人有空標

※ 這張表沿用 M2-2 的評估觀念：類別不平衡時（如「異常事件」只佔 2%），準確率沒有意義，要看 Recall 與 F1 。

完整解讀一次分類結果

任務：異常事件通報自動分類（是否為「用藥相關」） ｜ **測試集 1,000 筆，其中用藥相關 80 筆**

混淆矩陣

	預測：是	預測：否
實際：是	62	18
實際：否	45	875

綠色＝判對 ｜ 紅色＝判錯（18 ＝漏掉的，45 ＝誤報的）

指標

準確率 Accuracy	93.7%
精確率 Precision	57.9%
召回率 Recall	77.5%
F1	66.3%

★ 別被準確率騙了

全部猜「否」也有 92% 準確率（92% 本來就是「否」）

真正該看的是 Recall：

漏掉的 18 筆才是風險所在

模型學到哪些詞？（邏輯斯迴歸係數前十名 —— 這就是可解釋性）



★ 這張係數表才是你說服臨床單位的關鍵

「模型準確率 93%」說服不了人；「模型看的是給藥、劑量、藥名、點滴這些詞」可以 —— 它證明模型抓的是對的東西。

★ 別被準確率騙了

93.7% 看起來很好，但全部猜「否」也有 92%。
這是 M2-2 教過的類別不平衡陷阱，在文本分類特別常見。

該看 Recall

漏掉的那 18 筆才是風險所在。
醫療場域寧可誤報（45 筆）也不能漏報。

★ 係數表是關鍵

給藥、劑量、藥名、點滴——模型抓的是對的詞。
這證明它不是碰巧猜中。

這張表的真正用途

說服臨床單位。「模型準確率 93%」說服不了人，
「模型看的是這十個詞」可以。

★ 原生圖形，可直接在 PowerPoint 編輯

交付一份文本分類結果時，該附上什麼

最低限度（缺一不可）

- 混淆矩陣，不是只有準確率
- ★ Precision / Recall / F1 ，並說明你優先哪一個、為什麼
- 類別分布（讓人看得出不平衡程度）
- 模型看重的前十個詞或特徵
- 誤判案例：各挑 3–5 筆假陽性與假陰性，附原文

加分項（讓人信任你）

- 與基準線的比較（全猜多數類、關鍵字規則各是多少）
- 在不同子群的表現（不同科別、不同時期會不會掉）
- ★ 標註一致性（ Kappa ）—— 讓人知道天花板在哪
- 模型會在什麼情況下失效的誠實說明
- 後續怎麼監測：上線後準確率掉了誰會發現

★ 最後一項最常被忘記：語言會變、業務會變，今天準的模型半年後可能就不準了。

醫療文本分類的兩個實務提醒

① 類別幾乎總是不平衡

- 異常事件、跌倒、自殺意念——這些都是少數類別
- ★ 直接訓練會得到「全部判為正常」的高準確率廢物模型
- 處理方式沿用 M2-2：調整類別權重、過採樣、改看 Recall
- 醫療場域通常寧可誤報也不能漏報 → 優先拉高 Recall

② 要能解釋為什麼

- 「這份紀錄被判為高風險」——臨床人員一定會問為什麼
- 邏輯斯迴歸可以列出貢獻最大的前十個詞
- ★ 這種可解釋性在深度模型上難得多（M9 會深入）
- 實務折衷：模型給分數，同時列出命中的關鍵詞供人判讀

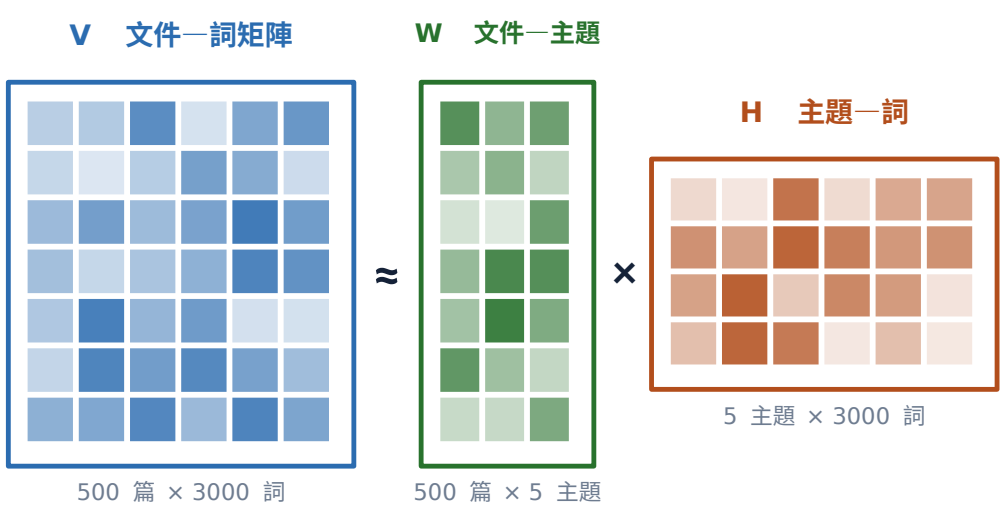
PART 06

主題模型

Topic Modeling

當你有一堆文字，但完全沒有標籤的時候。
這是文字版的分群——M3 的延伸。

NMF：從沒有標籤的文字中找出主題



跑出來長這樣（範例）

主題 1 → 疼痛管理
疼痛 止痛 不適 部位 加劇

主題 2 → 飲食衛教
飲食 鹽 血壓 控制 蔬果

主題 3 → 照顧者支持
家屬 照顧 負擔 喘息 服務

主題 4 → 傷口照護
傷口 換藥 引流 癒合 感染

★ 演算法只給你「詞的組合」，主題的名字是你取的

這一步需要領域知識——這正是護理背景的優勢：資工的人看到那組詞，不知道那代表什麼，你知道。

適用：問卷開放題想知道大家在講什麼、病人抱怨分類、衛教需求盤點——所有「有一堆文字但沒有標籤」的情況。

在做什麼

把文件—詞矩陣分解成「文件—主題」與「主題—詞」兩個矩陣。
概念上跟 M3 的 PCA 降維是同一類的事。

輸出是什麼

每個主題就是一組詞。演算法不會告訴你那組詞叫什麼。

★ 命名這一步需要領域知識

看到「疼痛 止痛 不適 部位 加劇」，
你知道那是疼痛管理——這正是護理背景的優勢。

適用情境

問卷開放題、病人抱怨分類、衛教需求盤點——
所有「有一堆文字但沒有標籤」的情況。

原講義： GIA 2. NLP - Natural Language Processing.pptx pp.58–61 ★ 原生圖形，可直接在 PowerPoint 編輯

主題模型的三種演算法與選用

演算法	適合資料量	特點	注意
NMF **非負矩陣分解**	< 1 萬篇	★ 結果穩定、可重現；輸入用 TF-IDF	主題數要自己指定（跟 K-Means 的 K 一樣）
LDA **潛在狄利克雷分配**	< 10 萬篇	機率模型，一篇文件可屬於多個主題	每次跑結果可能略有不同；輸入用詞頻不是 TF-IDF
BERTopic **嵌入式主題模型**	> 10 萬篇或短文本	★ 用嵌入向量，能處理短句與語意相近的表達	需要算力；解釋性較弱

★ 主題數怎麼決定：跟 M3 的分群一樣沒有標準答案。實務做法是跑 3 ～ 10 個主題數，人工看哪一組最說得通。

主題模型的三個常見誤用

① 資料量太小就跑

- 少於 200 篇文件，跑出來的主題多半是雜訊
- ★ 問卷開放題若只有 50 份回覆，直接人工讀完比較快也比較準
- 工具要配合資料規模，不要為了用而用

② 沒做好前處理就跑

- 停用詞沒去乾淨 → 每個主題都是「的、了、是」
- 專有名詞被切碎 → 主題裡全是沒意義的單字
- ★ 主題模型的結果品質，幾乎完全取決於前處理

③ 把主題當成客觀事實

- 主題是演算法找出的統計模式，不是「真的存在的分類」
- 換個主題數、換個演算法，結果就會變
- ★ 正確用法：當成「探索的起點」，幫你發現沒想到的面向，
而不是當成結論直接寫進報告

PART 07

情感分析與實體辨識

Sentiment Analysis & NER

兩個在醫護場域最實用的任務：
一個告訴你「病人怎麼想」，一個把敘述變成欄位。

情感分析：三種做法在同一句話上的差異

測試句：「這次住院的護理人員很細心，不過等待時間實在太久了。」

① 字典法
查情感詞典加總

- 細心 +1　　久－1　　太－1
- 總分 ＝ −1　→　判為「負面」

✘ 錯了。這句話對護理是正面的，只有等待時間是負面。字典法看不出對象。

② 傳統 ML
TF-IDF ＋ 邏輯斯迴歸

- 需要幾百筆已標註的評論訓練
- 輸出：負面 0.62 / 正面 0.38

△ 勉強。看得出整體偏負，但同樣分不出「對什麼」正負。

③ 面向情感分析
現代模型 ＋ 指定面向

- 面向「護理人員」→ 正面 0.94
- 面向「等待時間」→ 負面 0.91

★ 正確。這才是你真正需要的答案：知道哪個環節好、哪個環節該改。

★ 病人滿意度調查的開放題，用整句情感分析幾乎沒有價值

因為一句話裡通常同時誇跟罵。要做的是「面向情感分析」——分別看每一個面向的好惡。

字典法

查情感詞加總。這句話算出 −1 → 判為負面。
✘ 錯了：對護理是正面的，只有等待時間負面。

傳統 ML

TF-IDF ＋ 分類器。輸出整體偏負。
△ 勉強：同樣分不出「對什麼」正負。

★ 面向情感分析

「護理人員」正面 0.94 、「等待時間」負面 0.91 。
這才是你真正需要的答案。

★ 對滿意度調查的意義

一句話裡通常同時誇跟罵。
整句情感分析幾乎沒有價值——要做面向情感分析。

原講義： GIA 2. NLP - Natural Language Processing.pptx　pp.47–48　★ 原生圖形，可直接在 PowerPoint 編輯

命名實體辨識（NER）：把敘述變成欄位

輸入：一段門診紀錄（原始文字）

王小明 先生於 2027 年 3 月 5 日 至 臺北護理健康大學附設醫院 就診，診斷為

人名 PER　　日期 DATE　　機構 ORG

第二型糖尿病 ，處方 Metformin 500mg 每日兩次。

診斷 DIAG　　藥物 DRUG

輸出：可統計、可查詢的欄位

抽出的欄位	值
姓名 PER	王小明
日期 DATE	2027-03-05
機構 ORG	臺北護理健康大學附設醫院
診斷 DIAG	第二型糖尿病
藥物 DRUG	Metformin 500mg

通用 NER 模型認得的

- 人名、地名、機構名／日期、時間、數量
- 這些是預訓練模型的標準標籤

醫療 NER 要另外處理的

- ★ 診斷、藥物、劑量、檢驗項目
- 通用模型不認得 → 用醫療專用模型或少量標註微調

★ NER 是「病歷結構化」與「去識別化」共用的核心技術

把非結構化的敘述變成可以統計、可以查詢的欄位；也要先把人名、地址、病歷號找出來，才能遮蔽。

這是你做專題前必須先做的一步——不做去識別化，資料根本不能拿來分析。

通用模型認得的

人名、地名、機構、日期、數量——預訓練模型的標準標籤。

★ 醫療要另外處理

診斷、藥物、劑量、檢驗項目通用模型不認得，
要用醫療專用模型或少量標註微調。

用途一：病歷結構化

把非結構化敘述變成可統計、可查詢的欄位。

★ 用途二：去識別化

先把人名、地址、病歷號找出來才能遮蔽。
這是你做專題前必須先做的一步。

原講義： GIA 2. NLP - Natural Language Processing.pptx　pp.165–166　★ 原生圖形，可直接在 PowerPoint 編輯

現代方法：不用標註資料也能做的三件事

Hugging Face pipeline，三行程式碼起步

Python （課後自學；概念在 M7 深入）

```
from transformers import pipeline

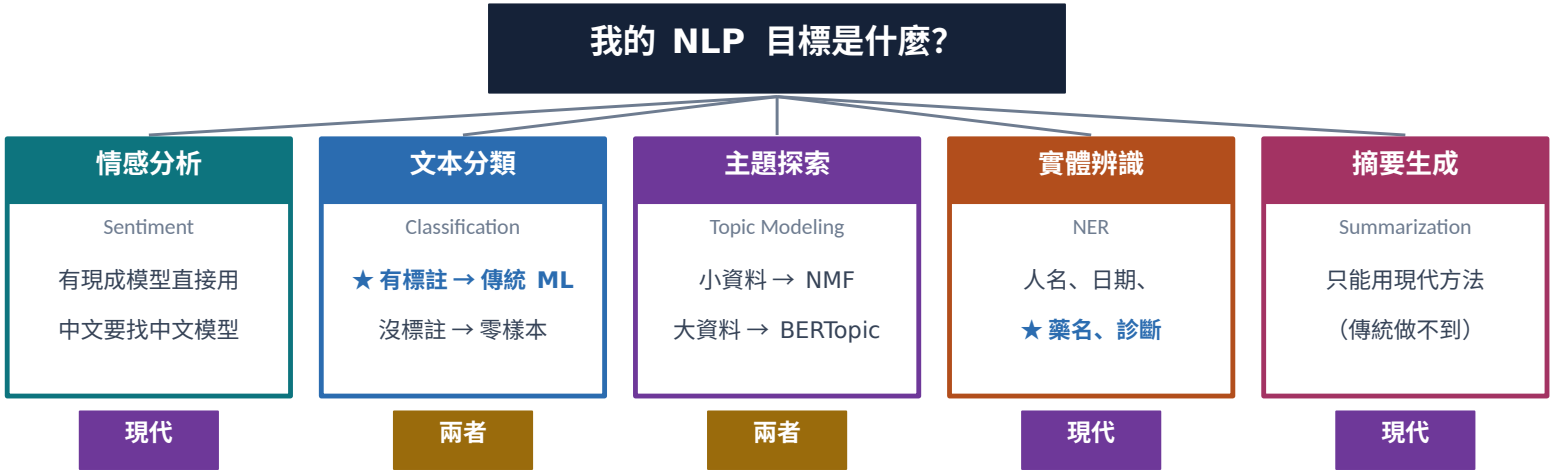
# ① 零樣本分類 Zero-shot：不需要任何標註資料
clf = pipeline('zero-shot-classification')
clf('病人反映等待領藥時間過長，希望能改善動線',
    candidate_labels=['醫療品質', '行政流程', '環境設施', '態度服務'])
# → （示意）行政流程 0.81 / 醫療品質 0.09 …；預設模型為英文，中文請用下方多語模型
#   ★ 你只要「想出類別名稱」，不用標任何一筆資料

# ② 摘要 Summarization：把長文變短
summ = pipeline('summarization')
summ(long_nursing_note, min_length=40, max_length=120)
# → 用於交班摘要、病程整理
#   ★ 注意：摘要是「生成」的，可能加入原文沒有的內容 → 必須人工覆核

# ③ 命名實體辨識 NER
ner = pipeline('ner', aggregation_strategy='simple')
ner('王小明先生於 2027 年 3 月 5 日至北護附設醫院就診')
# → [{'entity_group':'PER','word':'王小明','score':0.998}, ...]
#   ★ 去識別化的第一步就是這個

# ── 中文要指定中文模型，預設模型多為英文 ──────────────────
clf = pipeline('zero-shot-classification',
               model='joeddav/xlm-roberta-large-xnli')   # 多語言
ner = pipeline('ner', model='ckiplab/bert-base-chinese-ner')  # 中研院
```

任務地圖：哪個任務該用哪一代方法



什麼時候用傳統方法（TF-IDF ＋ ML）

- 資料量小（＜ 1 萬筆）且有標註
- **★ 需要可解釋：能指出是哪些詞造成判斷**
- 沒有 GPU、要跑在一般電腦上

什麼時候用現代方法（Transformer）

- 沒有標註資料（零樣本／少樣本）
- **★ 語序與上下文重要（如否定、條件句）**
- 任務本身需要生成（摘要、改寫）

★ 不要預設「新的一定比較好」

醫療場域要能解釋「為什麼判成這一類」的時候， TF-IDF ＋ 邏輯斯迴歸反而是對的選擇。

有標註 → 傳統方法夠用

資料量小、需要可解釋、沒有 GPU 時，TF-IDF ＋ 邏輯斯迴歸往往是對的選擇。

沒標註 → 現代方法

零樣本分類讓你「想出類別名稱」就能開始，

這在真實情境中價值極高。

語序重要 → 現代方法

否定句、條件句、委婉表達——詞袋處理不了。

★ 不要預設新的比較好

醫療場域要能解釋判斷依據時，傳統方法反而是正解。

原講義： GIA 2. NLP - Natural Language Processing.pptx　pp.45–46　★ 原生圖形，可直接在 PowerPoint 編輯

文件相似度與分群

五筆護理紀錄（已去識別化的虛構資料）

N1	傷口滲液增加，予以換藥
N2	傷口乾淨無滲液，換藥後覆蓋
N3	主訴傷口疼痛，給予止痛藥
N4	家屬詢問出院後照顧方式
N5	已完成出院衛教，家屬了解

分群結果（不需要任何標籤）

- 群 1： N1 、 N2 → 傷口照護／換藥
- 群 2： N4 、 N5 → 出院準備／家屬衛教
- N3 自成一群 → 疼痛管理（與傷口相關但主題不同）
- **整個過程只用到 TF-IDF 向量之間的距離**

餘弦相似度矩陣（1 = 完全一樣）

	N1	N2	N3	N4	N5
N1	1.00	0.82	0.41	0.05	0.08
N2	0.82	1.00	0.35	0.04	0.09
N3	0.41	0.35	1.00	0.06	0.11
N4	0.05	0.04	0.06	1.00	0.71
N5	0.08	0.09	0.11	0.71	1.00

綠色框＝自動浮出來的兩個群；顏色越深＝越相似

★ 這就是 M8 「知識檢索」的原型

只是那裡用嵌入向量、這裡用 TF-IDF —— 概念完全一樣：把問題與文件都變成向量，找最近的。

不需要任何標籤就能把一堆紀錄整理出結構，這是文字探勘最省力的一招。

怎麼算

把每份文件轉成 TF-IDF 向量，兩兩算餘弦相似度。

看得出什麼

N1 、 N2 相似度 0.82 （都在講傷口換藥）；
N4 、 N5 相似度 0.71 （都在講出院衛教）。

實務用途

找重複的紀錄、把幾千筆通報自動分群、找相似個案。

★ 這就是 M8 知識檢索的原型

只是那裡用嵌入向量、這裡用 TF-IDF 。
概念完全一樣：把問題與文件都變成向量，找最近的。

★ 原生圖形，可直接在 PowerPoint 編輯

標註品質：你的訓練資料本身可靠嗎

情境：兩位護理師各自標註同樣的 100 筆紀錄「是否為高風險」

	B：是	B：否	合計
A：是	35	10	45
A：否	5	50	55
合計	40	60	100

綠色＝兩人一致（85 筆）　紅色＝兩人不一致（15 筆）

Cohen's Kappa 怎麼算

觀察一致率　$P_o = 85/100 = 0.85$

期望一致率　$P_e = (45\times40+55\times60)/100^2$

$= (1800+3300)/10000 = 0.51$

$\kappa = (P_o - P_e)/(1 - P_e)$

$= (0.85-0.51)/(1-0.51) = 0.694$

★ 各自隨機標也有 51% 一致，Kappa 扣掉的正是這塊
（分級依 Landis & Koch, 1977）

< 0.20	0.21–0.40	0.41–0.60	0.61–0.80	0.81–1.00
極差	尚可	中等	**良好**	極佳

本例 κ = 0.694 → 落在「良好」；低於 0.60 就該先回頭修標註手冊，不要急著訓練模型

★ 監督式學習的天花板，是標註品質

兩個人標不一致的資料，模型不可能學得比人準。κ 太低不是換模型，是把「標註規則」寫清楚再重標一次。

為什麼要看這個

監督式學習的天花板是標註品質。
★ 兩個人標不一致的資料，模型不可能學得比人準。

Cohen's Kappa

扣掉「純靠運氣也會一致」的部分之後的真實一致率。
本例 κ = 0.694 → 良好。

★ 為什麼不能只看一致率

兩人各依自己的比例隨機標，也會有 51% 一致。
85% 聽起來很高，扣掉之後其實只有 0.694 。

κ 太低怎麼辦

不是換模型，是回頭把「標註規則」寫清楚，

再重新標一次。這一步省不掉。

★ 原生圖形，可直接在 PowerPoint 編輯

做標註的實務建議（上）：開始之前

① 先寫標註手冊，再開始標

- 每個類別要有定義、正例、反例，以及「模稜兩可時怎麼辦」
- ★ 最容易吵架的永遠是邊界案例——手冊要先把邊界講清楚
- 手冊本身就是研究方法的一部分，論文裡要交代

② 兩人獨立標一小批，先算 Kappa

- 先標 50–100 筆算 κ。低於 0.6 就先別繼續往下標
- 把不一致的案例拿出來討論，修訂手冊，再標一輪
- ★ 這比標完 1000 筆才發現標準不一致省事太多

這兩步是整個監督式文字分析中最容易被跳過、也最不該跳過的部分。

做標註的實務建議（下）：過程與收尾

③ 不一致的案例怎麼處理

- 找第三人仲裁，或兩人討論到達成共識
- 把仲裁結果回饋進手冊，避免同類型的爭議再吵一次
- 保留爭議案例清單——它們往往是最有研究價值的部分

④ 記錄標註者資訊與時程

- 誰標的、什麼時候標的、一次標了多久
- ★ 疲勞會顯著影響品質：連續標三小時的後段往往比較隨便
- 建議每次以 50 筆為一批，中間休息

※ 這一節的內容在護理研究的質性資料編碼上完全通用，不只用於機器學習。

PART 08

AI Studio 文字探勘實作

Hands-on with AI Studio

把前面所有概念在 AI Studio 裡實際跑一次。
本模組的範例是課程既有教材中最完整的一組。

課堂實作｜Amazon Alexa 評論的情感分析

★ AI Studio Repository / 7. Text Mining （需安裝 Text Processing 擴充）

開啟檔案

```
7. Text Mining/
  1 Creating Text objects.rmp
  1.2 amazon_alexa.csv
  7 Processing Text and Document Objects.rmp
  Case Study 1 Amazon_alexa
    sentiment analysis.rmp
  Case Study 3 Text mining -
    Lemmatizer TextClassification
    Sentiment .rmp
```

預期結果

- 前處理每開一個運算子，詞表會明顯變小
- TF-IDF 通常略優於單純詞頻
- 類別若不平衡，準確率會虛高
- Lemmatizer 對英文有效，對中文無效

操作步驟

1. 開啟「1 Creating Text objects」，看 Text 物件怎麼建立
2. 執行「7 Processing Text and Document Objects」
3. ★ 逐一開關 Tokenize ／ Filter Stopwords ／ Stem 運算子，觀察詞表變化
4. 執行 Case Study 1，看整體流程：讀檔 → 前處理 → 向量化 → 分類
5. 把 Process Documents 的向量化方式從 TF-IDF 改成 Term Occurrences，比較準確率
6. ★ 檢查混淆矩陣：正負評論的數量是否平衡？ Recall 各是多少？
7. Case Study 3 加了 Lemmatizer，比較它帶來多少提升

本模組的 AI Studio 範例對照

範例檔	示範什麼	對應本講義
1 Creating Text objects.rmp	建立 Text 物件、讀入文字資料	PART 2
7 Processing Text and Document Objects.rmp	★ 前處理運算子逐一示範	PART 2 、 PART 3
3. Web Mining from URLs and Tokenize.rmp	從網頁抓文字並斷詞	PART 2
4 / 5 / 6 Automatic Classification of Documents	文件分類與套用模型（含向量化版本）	PART 5
2 Find Text Association Rules.rmp	詞與詞的關聯規則	延伸
3 Document Similarity and Clustering.rmp	文件相似度與分群	PART 4 、 PART 6
Case Study 1 Amazon_alexa sentiment analysis.rmp	★ 完整情感分析流程	PART 5 、 PART 7
Case Study 2 IMDB with Process Documents	較大資料集的文本分類	PART 5
Spam Email Classification.rmp （4. Case Studies）	★ Naive Bayes 垃圾信分類，對應手算範例	PART 5

PART 09

醫療文本的四個特殊挑戰

Clinical Text: What Makes It Hard

通用 NLP 工具在醫療文本上會踩到的坑。

這一節是本模組最該記住的部分。

四個挑戰與對應做法

挑戰	實例	為什麼難	怎麼處理
否定與不確定	「否認胸痛」「疑似肺炎」 「無發燒」	詞袋模型丟掉語序， 「否認胸痛」與「胸痛」幾乎相同	★ 用否定偵測規則（如 NegEx ） 或改用理解上下文的現代模型
縮寫與同義詞	BP ／血壓／ Blood Pressure HTN ／高血壓	同一個概念多種寫法， 被當成不同的詞	建同義詞對照表，前處理時歸一
專有名詞被切碎	「二尖瓣脫垂」→ 二／尖／瓣／脫垂	一般詞典沒有收錄醫療術語	★ 自訂詞典（本模組 PART 3 ）
個資與去識別化	姓名、病歷號、地址、 聯絡電話	一旦外流無法收回； 法規與倫理要求	★ 先用 NER 找出來遮蔽， 再做任何分析

★ 第一列是最常被低估的：一個沒處理否定的醫療文本分類器，可能把「否認自殺意念」判成高風險，也可能反過來漏掉真正的個案。

隨堂練習：這是哪一種挑戰?

1 「否認胸痛」與「胸痛」在向量上幾乎一樣

否定與不確定

對策：否定偵測規則（NegEx）或懂上下文的模型

2 「疑似肺炎」被當成確診

否定與不確定

對策：不確定語氣也要標記

3 BP、血壓、Blood Pressure 被當成三個詞

縮寫與同義詞

對策：建同義詞對照表，前處理時歸一

4 「二尖瓣脫垂」被切成二／尖／瓣／脫垂

專有名詞被切碎

對策：自訂詞典

5 紀錄裡有病人姓名與病歷號

個資與去識別化

對策：先用 NER 找出來遮蔽或代換，再做任何分析

6 HTN 和高血壓分開計數

縮寫與同義詞

對策：同義詞歸一

四類挑戰：否定與不確定、縮寫與同義詞、專有名詞被切碎、個資與去識別化。通用 NLP 工具在醫療文本上會在這四處出錯——先遮住答案自己判斷。

去識別化：做專題之前必須先做的事

要移除或代換的十八類識別資訊（美國 HIPAA Safe Harbor）

- 姓名、地址、電話、傳真、 email 、身分證號、病歷號、健保卡號
- 帳號、憑證號碼、車牌、裝置序號、 URL 、 IP 、生物特徵、影像
- ★ 日期（只保留年）、 89 歲以上年齡合併；台灣另依個資法、人體研究法與 IRB

兩種做法

- 遮蔽（ Redaction ）：直接刪掉或換成 [姓名]、[日期]
- 代換（ Surrogate ）：換成假的但格式一致的值，保留可分析性
- ★ 研究用途多半選代換，因為要保留「同一個人」的關聯

★ 但去識別化不等於絕對安全

- 罕見疾病 + 特定年齡 + 就診地區，三者組合仍可能識別出個人
- 自由文本中的敘述（「本院唯一的某某手術個案」）也可能洩漏身分
- ★ 課程要求：一律使用公開資料集或完全虛構的資料，不碰真實病歷

實例：同一段病歷，標記、遮蔽、代換

① 標記實體

通用 NER 認得人名、日期、機構；診斷、藥物要用醫療專用模型或自訂規則。抽出來就能統計、查詢——這是病歷結構化。

王小明〔PER〕先生於 **2027 年 3 月 5 日**〔DATE〕至**臺北護理健康大學附設醫院**〔ORG〕就診，診斷為**第二型糖尿病**〔DIAG〕，處方 **Metformin 500mg**〔DRUG〕每日兩次。病歷號 **A123456**〔ID〕，聯絡電話 **0912-345-678**〔TEL〕。家屬**陳美華**〔PER〕女士陪同，主訴口渴、多尿。

② 遮蔽 Redaction

換成 [姓名]、[日期]。簡單安全，但失去「同一個人」的關聯。

[姓名] 先生於 **[日期]** 至 **[機構]** 就診，診斷為**第二型糖尿病**，處方 **Metformin 500mg** 每日兩次。病歷號 **[病歷號]**，聯絡電話 **[電話]**。家屬 **[姓名]** 女士陪同，主訴口渴、多尿。

③ 代換 Surrogate

換成格式一致的假值；同一個人在全文中換成同一個假名。研究用途多半選代換，以保留關聯；日期只保留年。

林志強先生於 **2027 年 [日期]** 至 **[某醫院]** 就診，診斷為**第二型糖尿病**，處方 **Metformin 500mg** 每日兩次。病歷號 **Z488031**，聯絡電話 **09XX-XXX-XXX**。家屬**張雅婷**女士陪同，主訴口渴、多尿。

虛構範例，規則式（正規表示式＋詞典）示範；實務可用 ckiplab/bert-base-chinese-ner 等模型。 HIPAA Safe Harbor 列出 18 類識別資訊；台灣另依個資法、人體研究法與 IRB 規定。

PART 10

醫護場域的應用案例

Applications in Healthcare

把前面所有技術組合起來，實際上能做什麼。

以下六個都是可以當成期末專題或碩論題目的方向。

六個可執行的應用方向

應用	用到的技術	資料來源	難度
病人滿意度開放題分析	★ 面向情感分析 ＋ 主題模型	院內滿意度調查（需去識別化）或公開評論資料	中
異常事件通報自動分類	斷詞 ＋ TF-IDF ＋ 分類器（或零樣本分類）	通報系統的敘述欄；可先用虛構資料建模	中
衛教需求盤點	主題模型 ＋ 詞頻分析	★ 衛教諮詢紀錄、線上問答平台的公開提問	低
護理紀錄品質稽核	文件相似度（找複製貼上）＋ 規則式關鍵欄位檢查	護理紀錄（需去識別化）	中
文獻回顧輔助	關鍵詞抽取 ＋ 文件分群 ＋ 摘要	★ PubMed 等公開資料庫，完全無個資疑慮	低
長照個案需求分析	NER （抽取需求項目）＋ 分群	★ 長照相關公開問答與法規文件	中

★ 標星號的三項資料完全公開，沒有個資問題，最適合當作期末專題或第一個練手的題目。

一個完整案例的想像：滿意度開放題分析

如果只看星等

- 「本月平均 4.2 分，比上月下降 0.3 分」
- 然後呢？你不知道是哪裡出了問題
- 主管會問「為什麼降？」，你答不出來
- ★ 星等只告訴你「幾分」，不告訴你「為什麼」

加上文字分析之後

- 主題模型跑出五個主題：等待時間、護理態度、環境、說明清楚度、費用
- 面向情感分析：護理態度 +0.91 、等待時間 − 0.87
- 時間趨勢：等待時間的負面比例從 12% 升到 31%
- ★ 結論具體到可以行動：問題在動線與排程，不在人員態度

這就是文字探勘的價值：把「一個分數」變成「一份可以據以改善的診斷」。

開始一個文字探勘專題前，先問自己五個問題（上）

① 資料拿得到嗎？合法嗎？

- ★ 這是第一個問題。拿不到或不能用，後面全是空談
- IRB 、資料使用同意、去識別化——這些通常比分析更花時間
- 課程建議：先用公開資料把方法跑通，再談院內資料

② 資料量夠嗎？

- 分類至少要幾百筆有標註的；主題模型至少要幾百篇
- 少於 100 筆，人工讀完比較快也比較準
- ★ 這不是失敗，是務實。工具要配合資料規模

這兩題決定「這個題目做不做得成」。過不了就換題目，不要硬做。

開始一個文字探勘專題前，先問自己五個問題（下）

③ 我要的答案，真的需要文字分析嗎？

- 有時候一個結構化欄位就能回答，不必大動干戈
- ★ 先問「這個答案會改變什麼決策」，再決定要做多複雜

④ 結果要給誰看？他看得懂嗎？

- 給臨床單位看的東西要能解釋為什麼——這會影響你選哪一代方法
- ★ 一個準確率 92% 但說不出理由的模型，在臨床往往推不動

⑤ 錯了會怎樣？

- 高風險場景（自殺意念、異常事件）要留人工覆核，不能全自動
- 設計時就要想好：模型判錯的那幾筆，誰會發現、怎麼補救

★ 這五題不只適用於文字探勘。任何資料科學專案開始前都值得問一遍。

從 TF-IDF 到詞向量： M6 與 M7 的分界線

TF-IDF 的世界（M6）

「醫師」與「醫生」是兩個完全無關的欄位



每個詞是矩陣裡的一欄，彼此之間沒有「距離」的概念

詞向量的世界（M7 預告）

語意相近的詞，向量位置也相近



距離近＝意思近；同義詞自然聚在一起

詞向量最著名的性質：可以做「運算」

vec(國王) − vec(男) + vec(女) ≈ **vec(女王)**

vec(醫師) − vec(醫院) + vec(學校) ≈ **vec(教師)（示意）**

vec(巴黎) − vec(法國) + vec(日本) ≈ **vec(東京)**

這代表向量真的捕捉到了語意結構，不只是統計共現。

★ Word2Vec （2013）是這條路的起點； M7 的 Embedding 與 Transformer 就是它的後代

M6 到此為止：文字已經變成電腦能算的東西。下一步是讓那些數字帶有「語意」。

TF-IDF 的世界

「醫師」與「醫生」是兩個完全無關的欄位。

彼此之間沒有「距離」的概念。

詞向量的世界

語意相近的詞，向量位置也相近。
「醫師」與「醫生」會靠在一起。

★ 最著名的性質

vec(國王) − vec(男) + vec(女) ≈ vec(女王)。
這代表向量真的捕捉到語意結構，不只是統計共現。

Word2Vec （2013）是起點

M7 的 Embedding 與 Transformer 都是它的後代。
下週接著講。

★ 原生圖形，可直接在 PowerPoint 編輯

三種文本表示法的完整對照（本模組總結）

	詞頻／TF-IDF （M6）	詞向量 Word2Vec	上下文向量 BERT （M7）
維度	幾千～幾萬（稀疏）	300 左右（稠密）	768 以上（稠密）
同義詞	★ 視為完全無關	位置相近	位置相近
一詞多義	無法區分	★ 無法區分（一詞一向量）	★ 可區分（依上下文變化）
語序	丟失（N-gram 可部分補救）	丟失	★ 完整保留
可解釋性	★ 高：每一維就是一個詞	中	低
算力需求	★ 極低	低	高
什麼時候用	小資料、要解釋、要稽核	詞層級的語意任務	語意與上下文重要時

★ 這張表把 M6 與 M7 串起來了。注意最後兩列：不是越新越好，是看你的限制條件在哪。

M6 → M7 → M8：同一條線的三個階段

M6 （本週）
傳統文字探勘

- 斷詞、去停用詞、正規化
- 詞袋與 TF-IDF
- Naive Bayes 分類
- NMF 主題模型

★ 可解釋、可稽核、不用 GPU

這一階段的文字表示法
詞袋 Bag of Words

→

M7 （下週）
生成式 AI 原理

- Tokenization （斷詞的進階版）
- Embedding （TF-IDF 的進階版）
- Attention 與 Transformer
- 預訓練／微調／推論

★ 懂上下文，但不易解釋

這一階段的文字表示法
向量 Embedding

→

M8 （第 13 週起）
Dify 應用開發

- RAG 知識庫（就是文件檢索）
- 切塊（就是文本前處理）
- 召回（就是相似度計算）
- Agent 與工具調用

★ 期末專題

這一階段的文字表示法
語意空間 + 檢索

★ 三週是同一條線：都在處理「文字怎麼變成電腦能算的東西」

只是表示法一路變聰明：詞袋（M6）→ 向量（M7）→ 語意空間與檢索（M8）。

所以這一週學的前處理、相似度、評估觀念，到 M8 做期末專題時還會再用一次。

M6 （本週）
詞袋與 TF-IDF。可解釋、可稽核、不用 GPU。

M7 （下週）
Tokenization 是斷詞的進階版，Embedding 是 TF-IDF 的進階版。
懂上下文，但不易解釋。

M8 （第 13 週起）
RAG 就是文件檢索、切塊就是文本前處理、召回就是相似度計算——全是這一週學過的東西。

★ 三週是同一條線
都在處理「文字怎麼變成電腦能算的東西」。表示法從詞袋 → 向量 → 語意空間，一路變聰明。

★ 原生圖形，可直接在 PowerPoint 編輯

Take-Home Assignment ｜ M6 文字探勘

W12 上課前繳交 · 不另計分（隨堂 Lab）

作業說明

【任務一】前處理的影響實驗 （40%）
資料：AI Studio 的 amazon_alexa.csv（或自選公開中文評論資料集）

1. 建立一個文本分類流程：讀檔 → 前處理 → TF-IDF → Naive Bayes
2. 做四組實驗，其他條件固定，只改前處理：
   (a) 完全不做前處理 (b) 只斷詞 (c) 斷詞＋去停用詞
   (d) 斷詞＋去停用詞＋詞形還原
3. 記錄四組的準確率、Recall、F1，畫成一張表
4. 寫 200 字：哪一步帶來的提升最大？有沒有哪一步反而變差？

【任務二】中文斷詞與自訂詞典 （35%）

1. 自己寫 10 句含醫護專有名詞的句子（虛構，不得用真實病歷）
2. 用 jieba 預設詞典斷詞（Python，可用 Colab），記錄哪些術語被切碎
3. 建立一份自訂詞典（至少 15 個詞），重新斷詞
4. 做成前後對照表，寫 150 字說明差異

【任務三】主題模型探索 （25%）

1. 找一份公開的中文評論或問卷開放題資料（至少 300 筆）
2. 跑 NMF（Python）或 AI Studio 的 LDA，主題數分別設 3、5、8
3. 為每個主題人工命名，說明你的命名依據
4. 寫 150 字：哪一個主題數最說得通？為什麼？

【繳交】PDF 一份，含流程截圖與結果表。檔名：M6_學號_姓名.pdf
【提醒】★一律使用公開資料集或虛構資料，不得使用任何真實病歷或個案紀錄。

任務一紀錄表：四組前處理實驗

設定	準確率	Recall	F1	與前一組相比（F1）
(a) 完全不做前處理				—
(b) 只斷詞				
(c) 斷詞＋去停用詞				
(d) 斷詞＋去停用詞＋詞形還原				

其他條件固定，只改前處理。若某一步反而變差，想想為什麼——例如停用詞表刪掉了否定詞?
找出提升最大的一步，寫進那 200 字。

填入四組實驗的結果（0–1）。看 Recall 與 F1，不要只看準確率——醫療文本的類別幾乎總是不平衡。網站版可直接填寫並下載 CSV。

本模組重點回顧

1. 醫護領域最有價值的資料多半是文字，而你有看得懂它的優勢
2. NLP 三個時代並存：醫療今天仍大量使用規則式與 TF-IDF ，因為可稽核
3. ★ 大部分時間花在前處理。中文斷詞做不好，後面全白做
4. 自訂詞典是最划算的投資——效果通常大於換一個更厲害的模型
5. TF-IDF 讓稀有詞浮出來：「病人」沒有鑑別力，權重最低
6. Naive Bayes 「天真」但好用；小資料的文本分類首選

M6 名詞速查表　NLP 與文字探勘

中文	English	一句話說明
自然語言處理	Natural Language Processing, NLP	讓電腦處理與理解人類語言的技術領域
文字探勘	Text Mining	從非結構化文字中萃取有用資訊與模式
語料庫	Corpus	用於分析或訓練的文本集合
斷詞／分詞	Word Segmentation / Tokenization	把連續文字切成詞；中文因無空格而特別困難
自訂詞典	User Dictionary	補充斷詞器未收錄的專有名詞，醫療文本必備
停用詞	Stop Words	無實質意義的高頻詞；注意否定詞常誤列其中
詞形還原	Lemmatization	把詞還原為原形（running → run）；主要用於英文
詞幹提取	Stemming	以規則截去詞尾；較粗糙，一般建議改用詞形還原
詞性標註	Part-of-Speech Tagging, POS	標記名詞、動詞等詞性，可用於篩選

M6 名詞速查表　NLP 與文字探勘

（2 / 4）

中文	English	一句話說明
向量化	Vectorization	把文字轉成數值形式，供機器學習使用
詞袋模型	Bag of Words, BoW	只計次數、不管語序的文本表示法
文件—詞矩陣	Document-Term Matrix, DTM	列為文件、欄為詞的矩陣，值可為次數或權重
詞頻	Term Frequency, TF	某詞在該文件中出現的頻率
逆文件頻率	Inverse Document Frequency, IDF	出現在越少文件的詞權重越高
TF-IDF	Term Frequency–Inverse Document Frequency	詞頻乘以逆文件頻率，傳統文字探勘的標準權重
稀疏矩陣	Sparse Matrix	絕大多數元素為 0 的矩陣； DTM 的典型型態
N-gram	N-gram	連續 N 個詞的組合，可部分保留語序資訊
樸素貝氏	Naive Bayes	假設特徵條件獨立的分類器；文本分類上表現優異

M6 名詞速查表　NLP 與文字探勘

（3 / 4）

中文	English	一句話說明
拉普拉斯平滑	Laplace Smoothing	分子加 1、分母加 k，避免機率為零
主題模型	Topic Modeling	從無標籤文本中找出潛在主題
非負矩陣分解	Non-negative Matrix Factorization, NMF	把 DTM 分解為文件—主題與主題—詞兩矩陣
潛在狄利克雷分配	Latent Dirichlet Allocation, LDA	機率式主題模型，一文可屬多主題
情感分析	Sentiment Analysis	判斷文本的情緒傾向
面向情感分析	Aspect-Based Sentiment Analysis, ABSA	★ 針對特定面向分別判斷情緒，滿意度分析必備
命名實體辨識	Named Entity Recognition, NER	找出並標記人名、地名、日期、藥名等實體
否定偵測	Negation Detection	辨識「否認」「無」等否定語境；醫療文本關鍵
去識別化	De-identification	移除或代換可識別個人身分的資訊

M6 名詞速查表　NLP 與文字探勘

（4 / 4）

中文	English	一句話說明
零樣本分類	Zero-shot Classification	不需標註資料，只給類別名稱即可分類
少樣本學習	Few-shot Learning	僅用少量範例即可完成任務
摘要生成	Summarization	把長文本濃縮成短摘要；屬生成式任務需人工覆核
文件相似度	Document Similarity	以向量距離衡量兩份文件的相近程度
餘弦相似度	Cosine Similarity	以向量夾角衡量相似度，文本分析常用
jieba	jieba	常用中文斷詞套件，輕量易擴充，簡體語料為主
CKIP Transformers	CKIP Transformers	中研院開發，以臺灣繁體語料訓練，含斷詞、詞性與 NER
spaCy	spaCy	主流的英文 NLP 工具庫，提供完整前處理流程
Hugging Face Pipeline	Hugging Face Pipeline	以數行程式碼呼叫預訓練模型完成常見 NLP 任務

課後複習：本單元的互動學習站

掃描 QR code 或輸入網址；手機、平板、電腦都能用

掃我開啟 M6 互動版

手機相機對準即可開啟

網址

nchsieh.github.io/assets/courses/001/interactive/m6/

20 節，順序對應講義各 PART

看完一個 PART 就做對應的那一節，不必一次做完

改參數、立刻看結果

圖表可以拖拉調整，看懂「為什麼」，而不只是背結論

隨堂練習＋即時回饋

答錯會說明原因；作答進度只存在你自己的瀏覽器

全部單元入口： nchsieh.github.io/assets/courses/001/interactive/ · 網站內容為教學示例與模擬資料，不需登入